Arithmetic Circuits

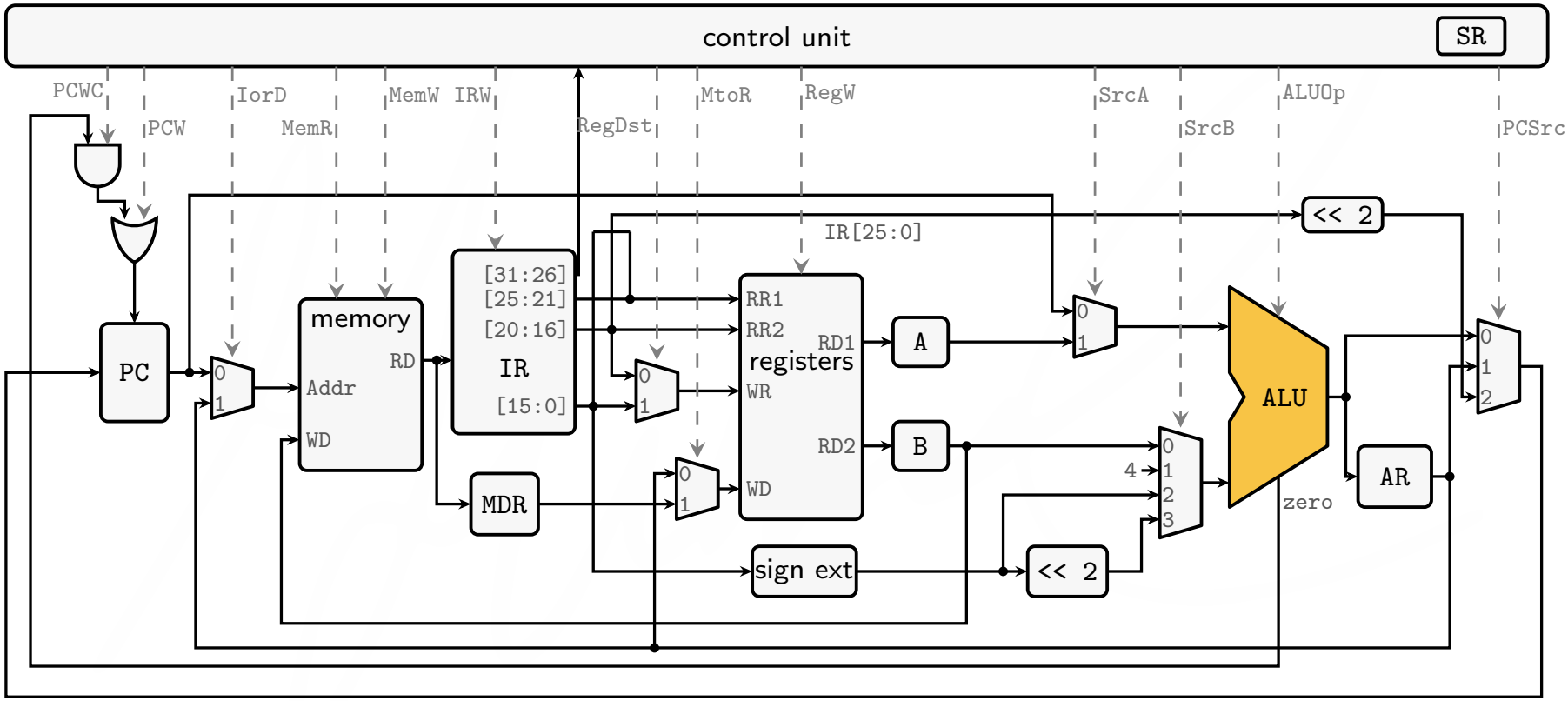
Computer Organization

Chandra Gummaluru

chandra-gummaluru.github.io



We now want to build the part of the computer that does the **arithmetic**.

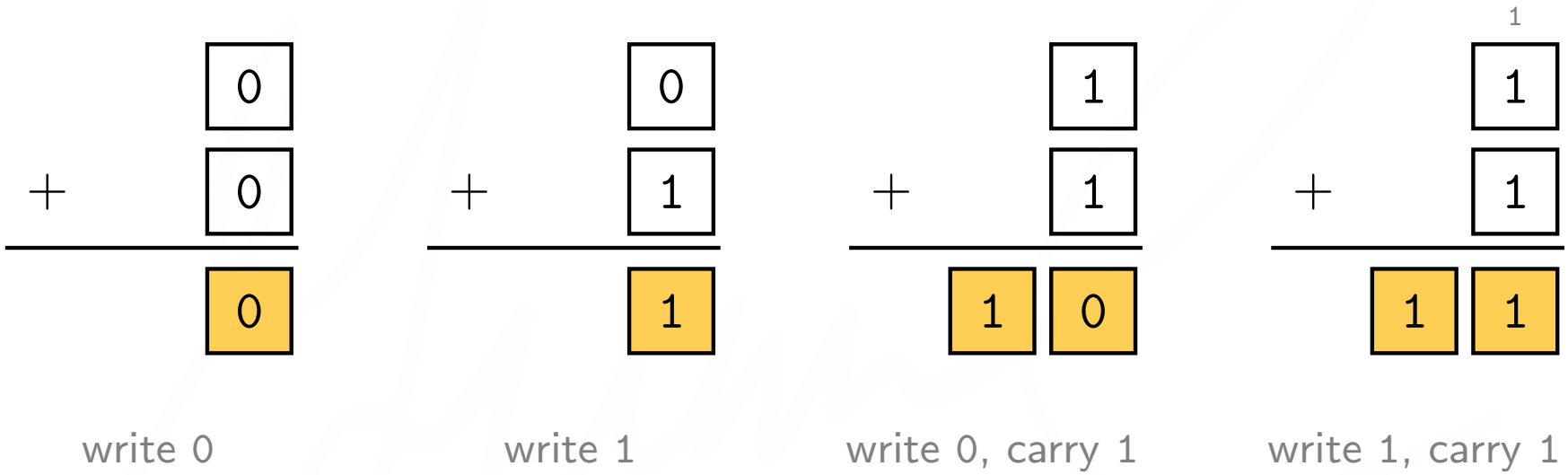


To perform arithmetic on binary numbers, we need to fix the number of bits the system uses to represent numbers, n.

For illustrative purposes, let's assume a 4-bit system.

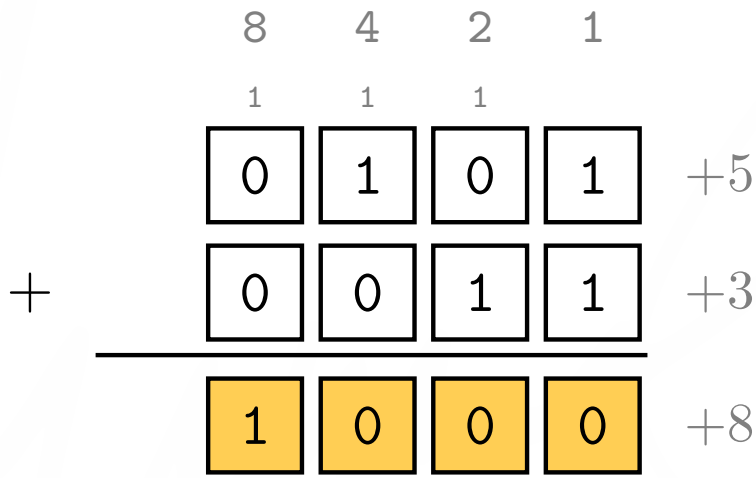
addition

We add binary numbers one column at a time, from the right, just as in decimal. A column can only ever be one of four cases.



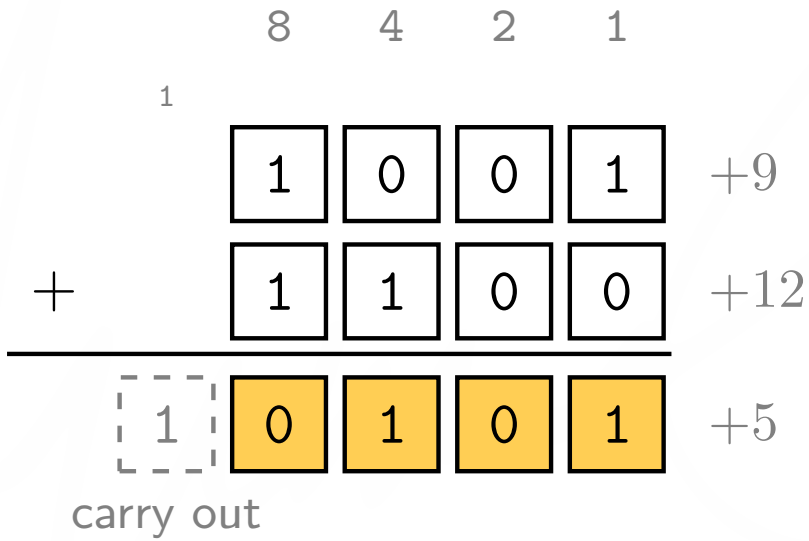
A column adds at most three bits (the third is a carry in), so it always gives one bit to write and at most one to carry.

To add 0101 (5) and 0011 (3), we work right to left, writing each column's carry above the next.



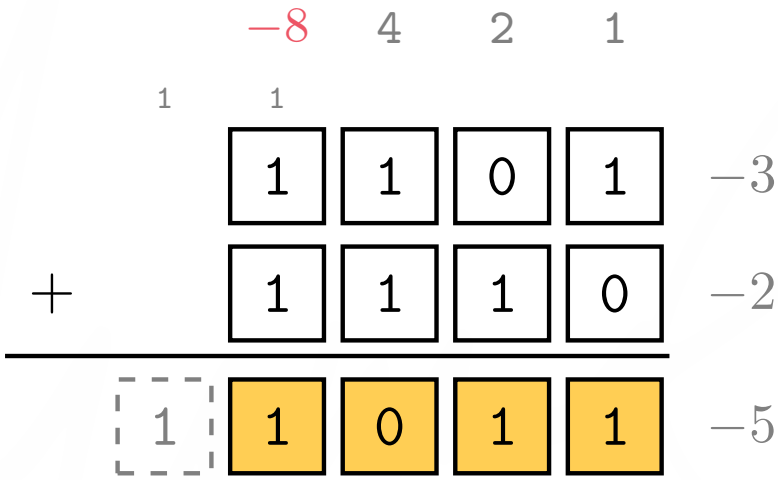
adding $5 + 3$ in 4-bit unsigned binary

With n bits, a sum can need $n + 1$. The extra bit is the **carry out**; the result keeps only n bits, so the carry out is lost.



adding $9 + 12$ in 4-bit unsigned binary

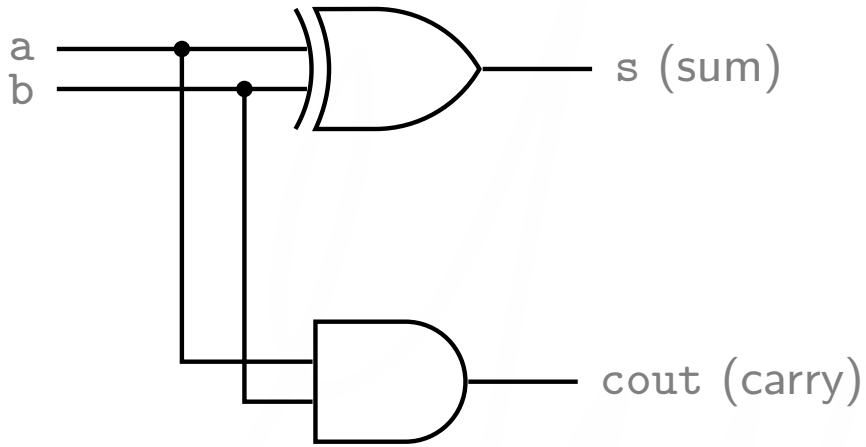
Signed addition works in exactly the same way.



adding $-3 + (-2)$ in 4-bit signed binary

adders

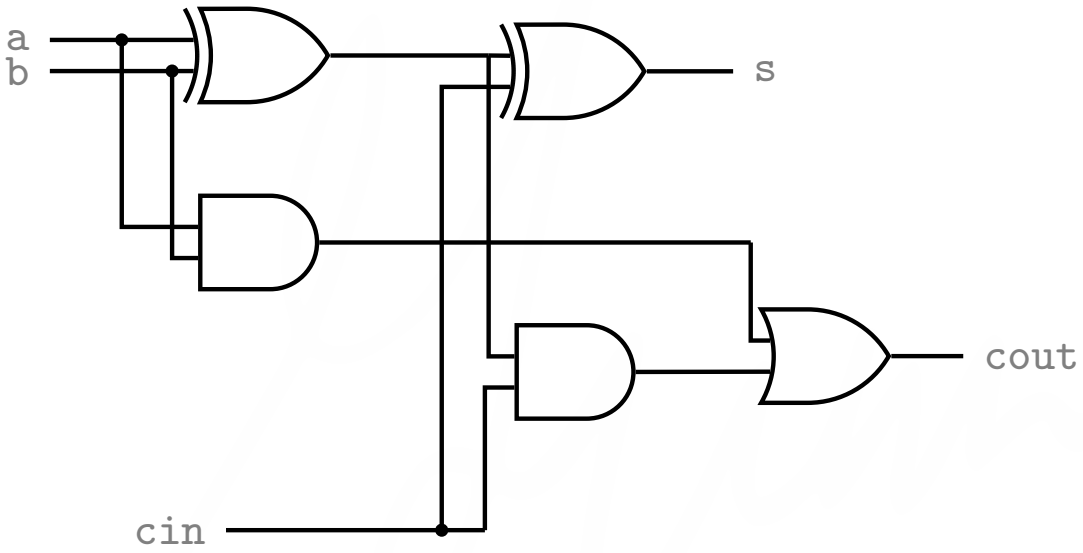
A **half-adder** adds two 1-bit inputs. It produces a sum bit and a carry bit.



a	b	s	cout
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

To add n-bit inputs, we need to be able to send the carry from one adder as an input of another.

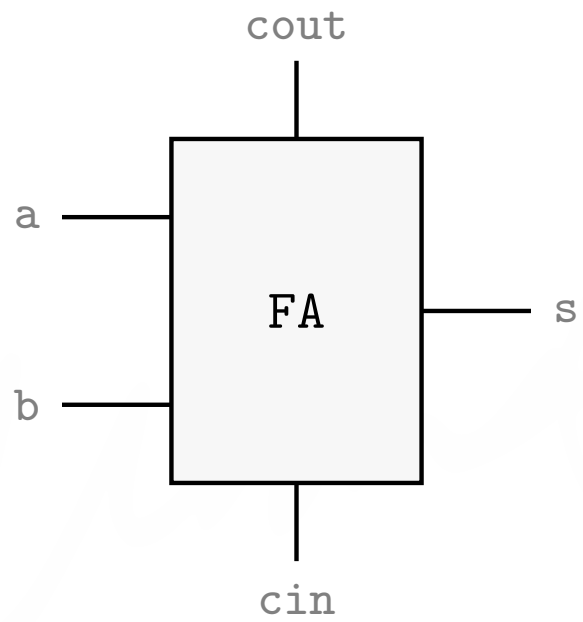
Wiring the two half-adders together gives a **full-adder** circuit. The first adds a and b, the second adds a carry in.



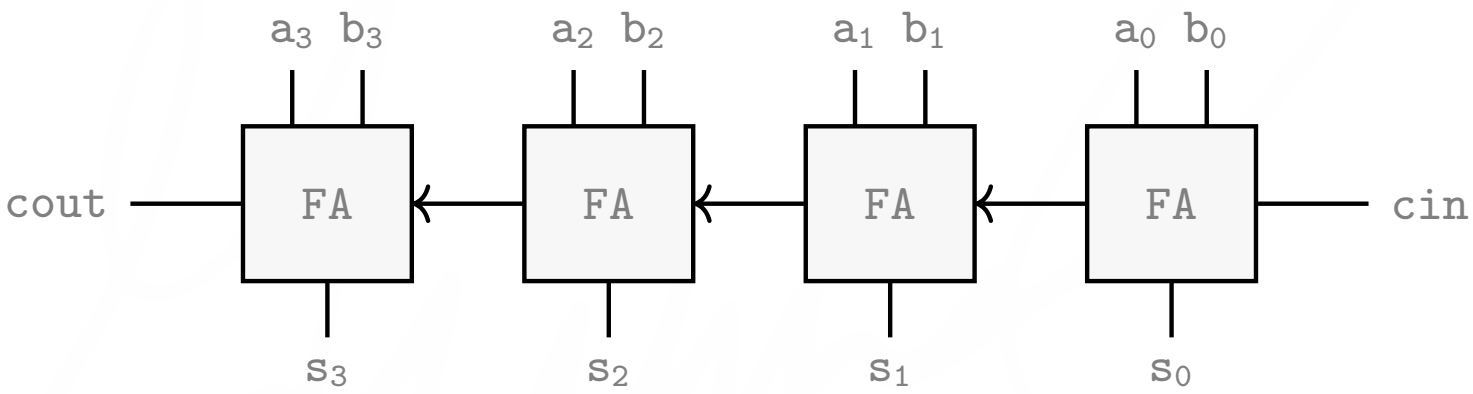
a	b	cin	s	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

This lets us chain adders together.

The resulting device looks like this:

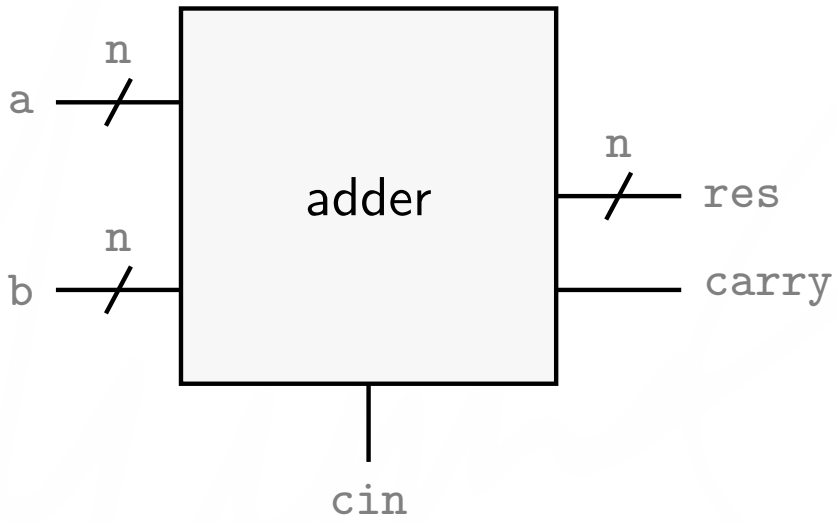


We chain full-adders so the carry out of one bit feeds the carry in of the next. This adds two whole numbers.



the carry ripples from the lowest bit to the highest

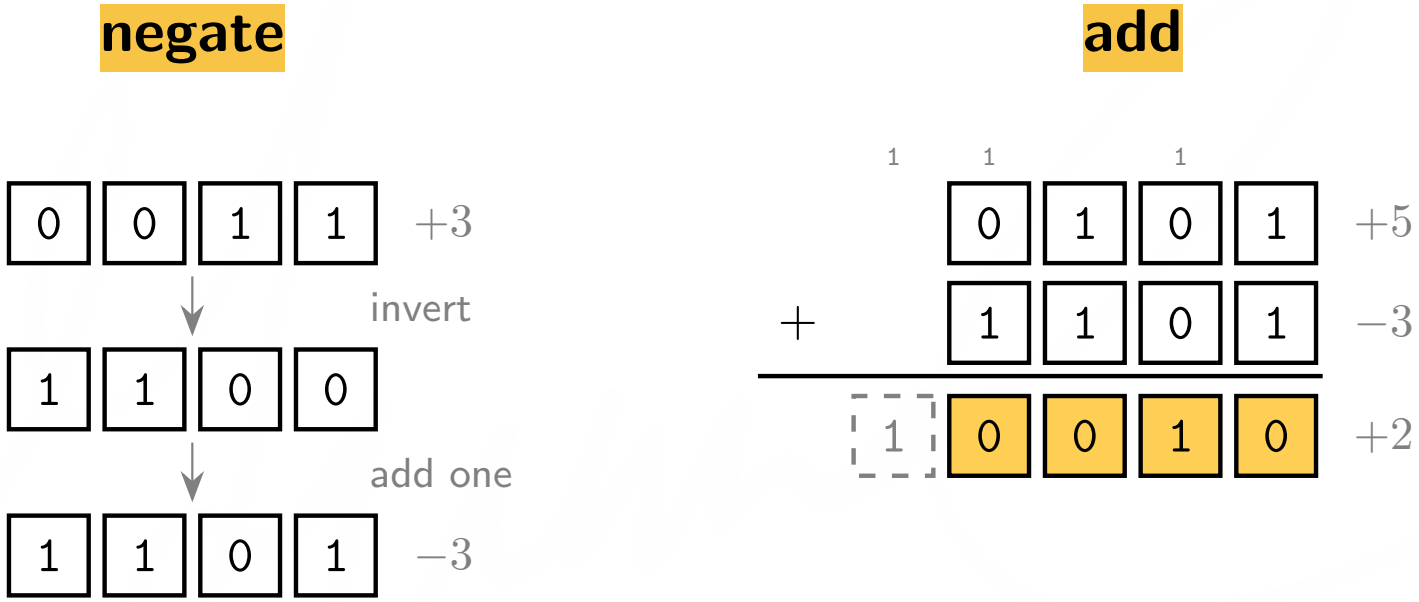
The resulting device looks like this:



We typically wire `cin` to 0.

subtraction

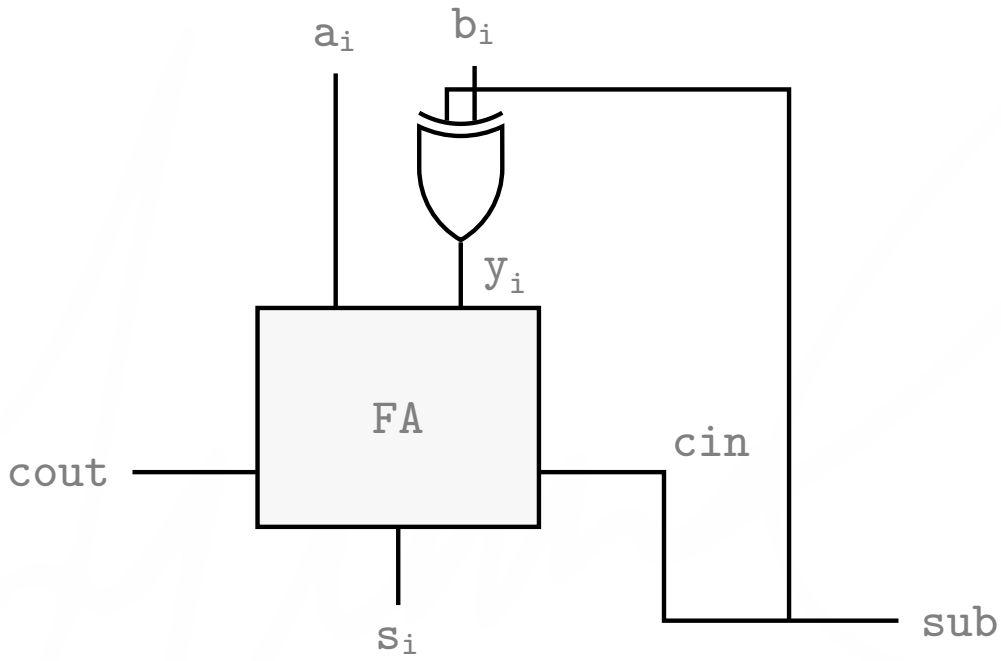
To compute 0101 − 0011 (5 − 3), we first negate 0011, then add the result to 0101.



subtracting $5 - 3$ in 4-bit binary; the carry out is dropped, as in any addition

subtractors

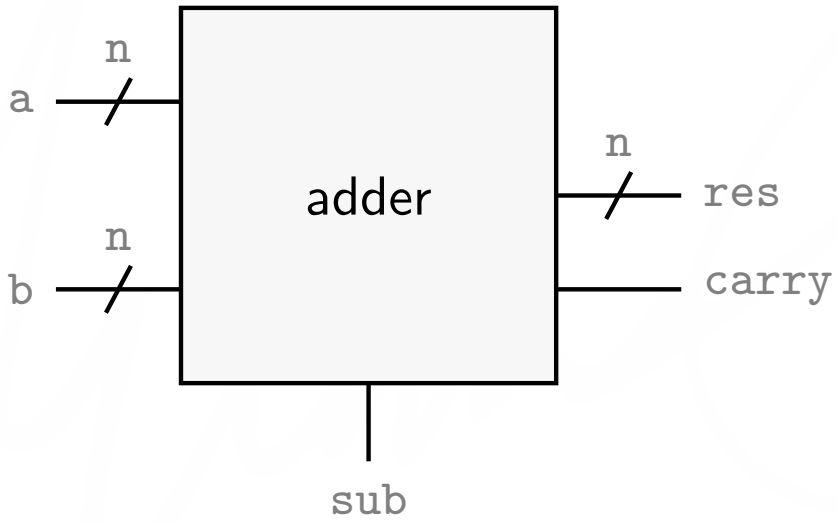
To compute `a - b`, we add the negation of `b`, which means inverting its bits and then adding one.



this slice is repeated for every bit, sharing the one `sub` line

The bit-inversion can be done using an `XOR`. The `+1` can be added separately using `cin`.

The same adder now does both operations. One control bit selects whether to add or subtract.



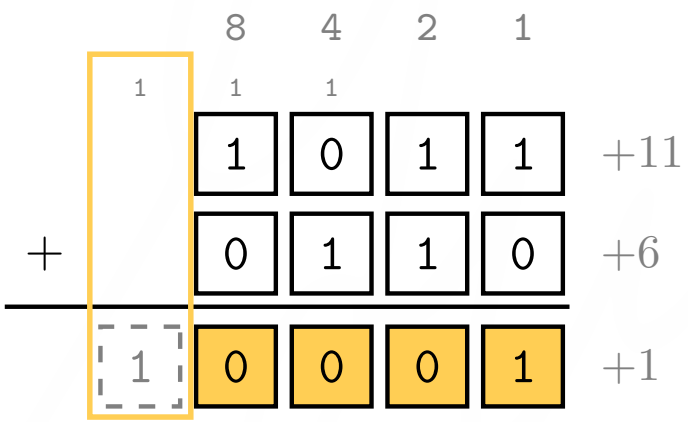
overflow

A fixed-width number can only hold values within a certain range. If a result falls outside that range, it has **overflowed**.

How we spot it depends on what we compute.

unsigned addition

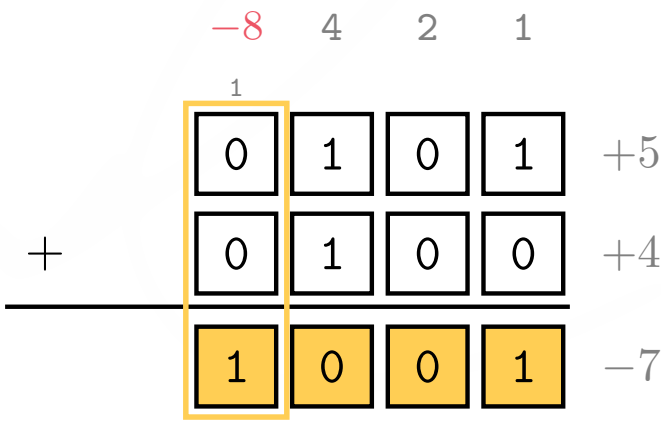
adding $11 + 6$



$\texttt{cout} = 1$

signed addition

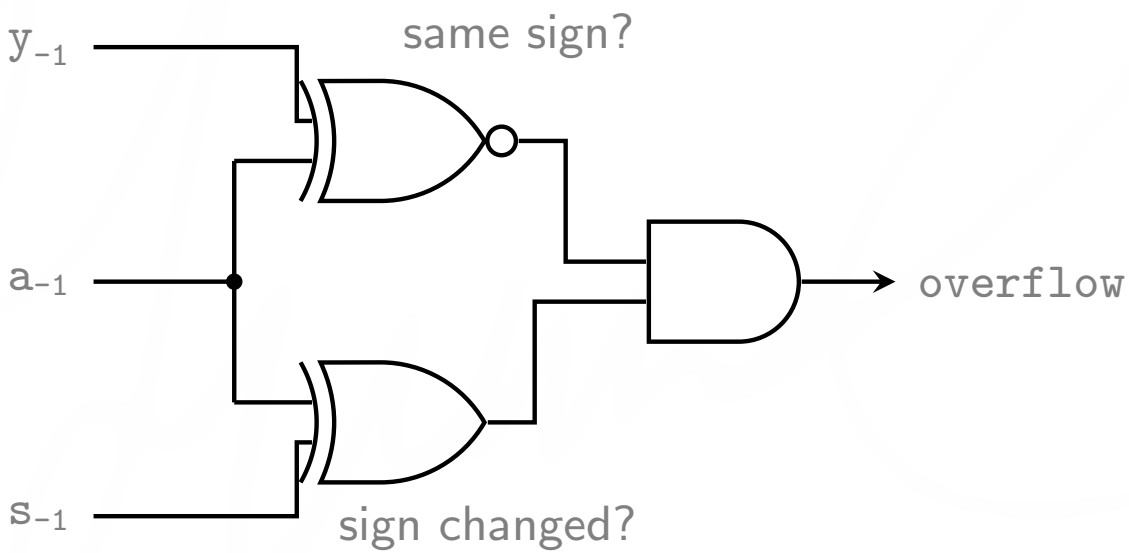
adding $5 + 4$



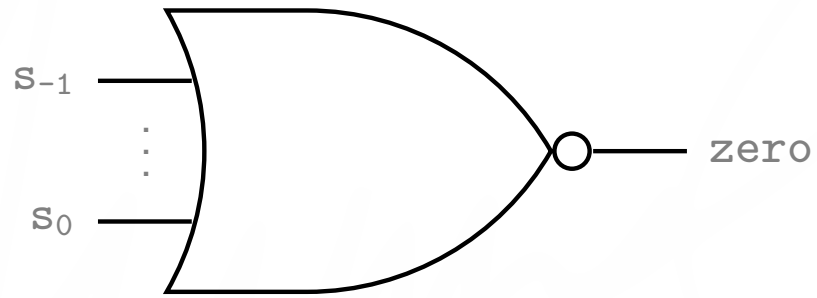
$\texttt{a}_{-1} = \texttt{b}_{-1} \neq \texttt{s}_{-1}$

(in signed addition, if a and b are different signs, overflow cannot occur)

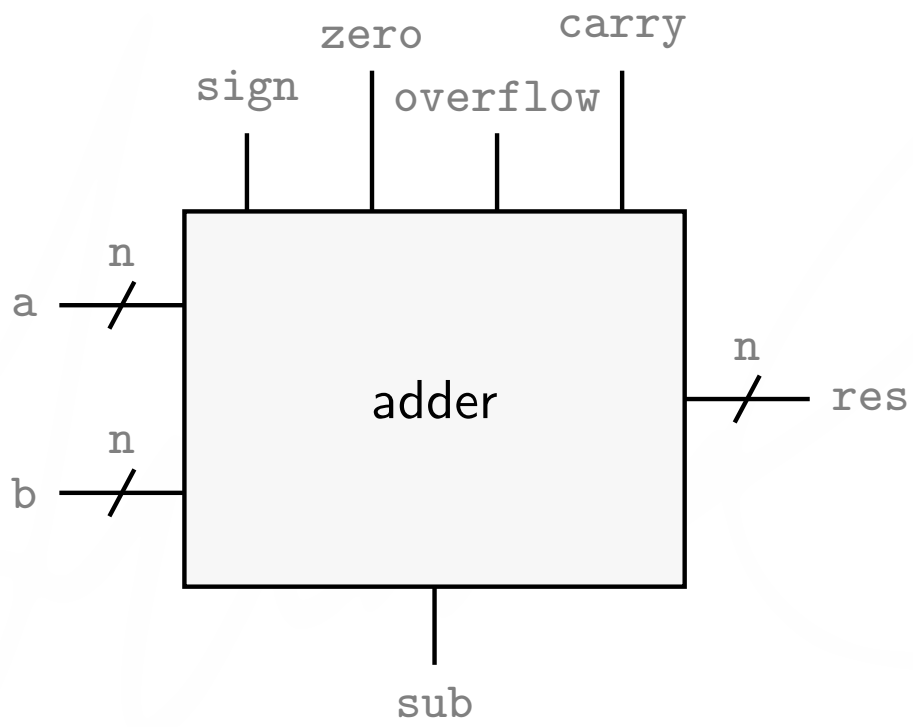
The adder reports both checks as **flags**: `carry` for unsigned numbers, `overflow` for signed ones.



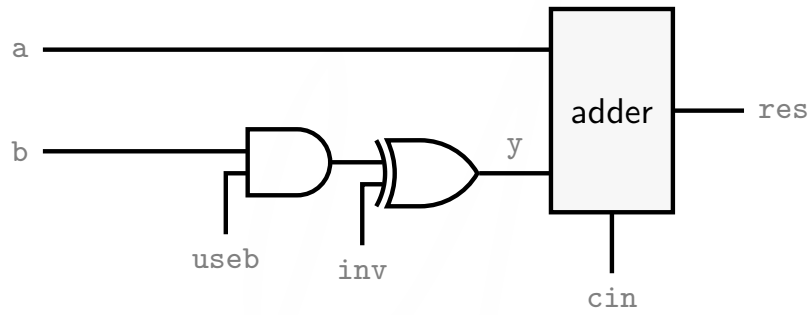
A `zero` flag is set when every bit of the result is `0` — equivalently, when none of the bits are `1`. An `n`-input `NOR` of the result bits does exactly this.



Putting it all together gives us an adder with its flags.



We can use the adder to perform other useful operations by choosing what it adds to a using three control bits.

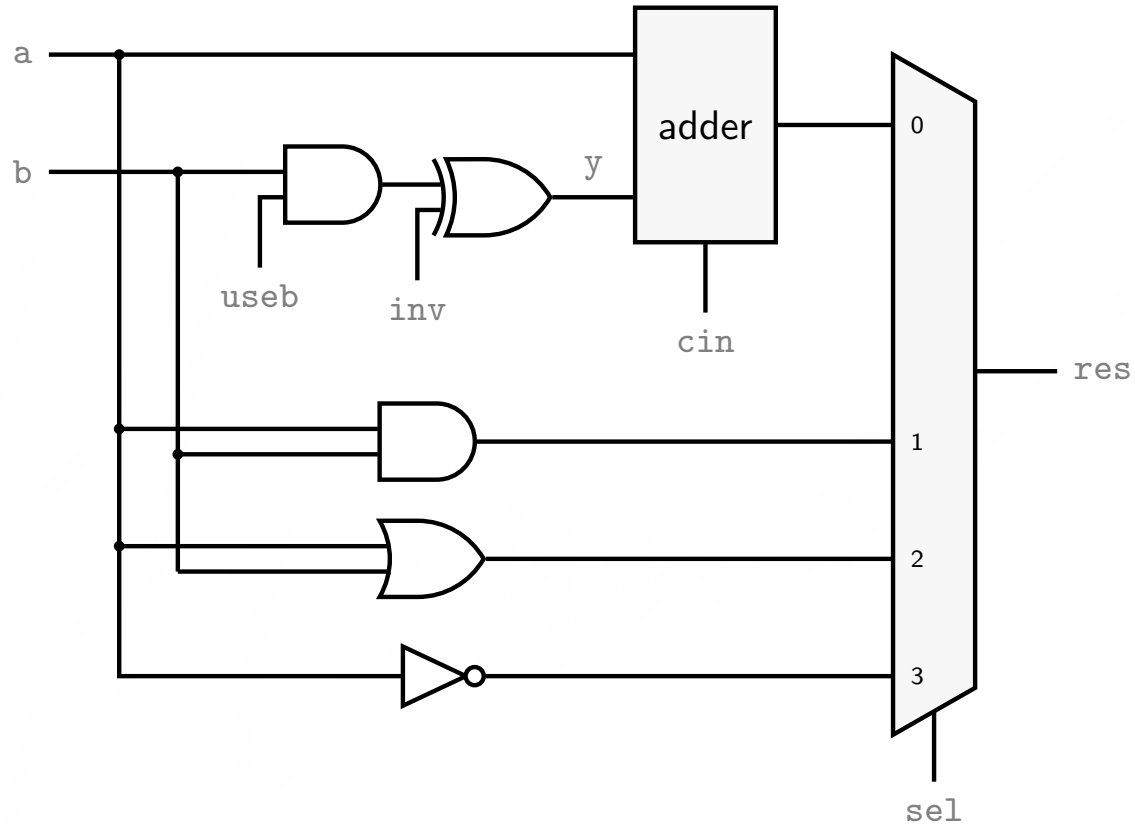


	useb	inv	cin	y
transfer	0	0	0	0
increment	0	0	1	0
decrement	0	1	0	1...1
add	1	0	0	b
subtract	1	1	1	$\overline{\texttt{b}}$

The control bits are:

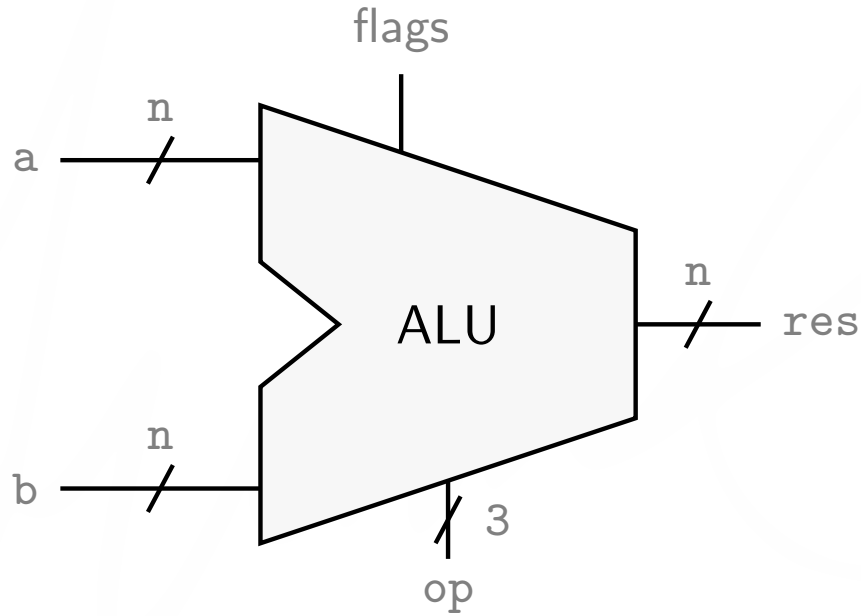
- `useb`: whether we should use `b` or use `0` in lieu of `b`
- `inv`: whether we should invert `b` or not
- `cin`: whether we should include a carry in or not

We can also throw in some (bit-wise) logical operations. Putting it together gives us an **arithmetic logic unit (ALU)**.



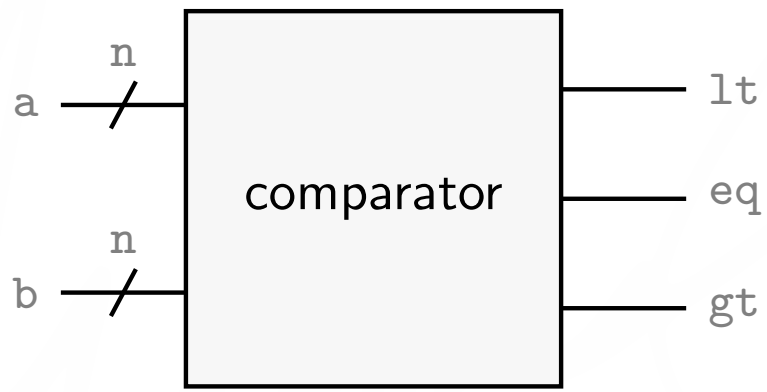
the control bits `sel`, `useb`, `inv` and `cin` choose the operation

The resulting block looks like this, where op selects the operation:



comparators

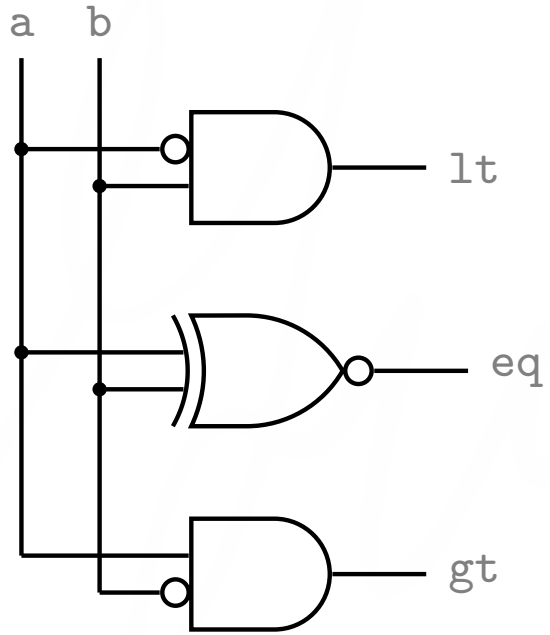
A **comparator** takes two numbers and tells us how they are ordered: exactly one of `a < b`, `a = b`, `a > b` holds.



each output is a single bit; exactly one of them is 1

We could build a comparator using an adder (with subtraction), but this would be too slow. A better approach could directly compare each bit.

If a and b are one bit each, then we can derive the following relationship:

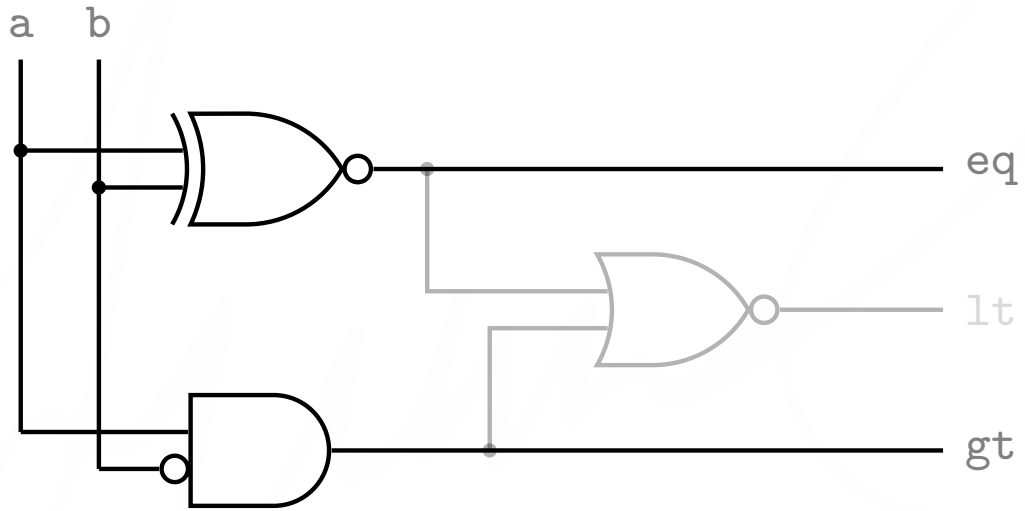


a	b	lt	eq	gt
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

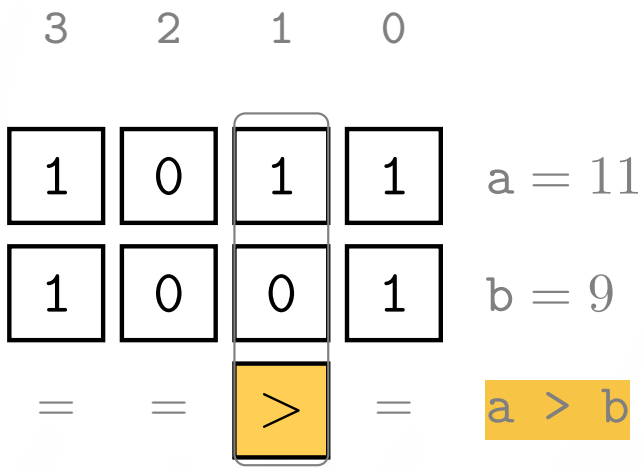
$\mathtt{lt} = \overline{\mathtt{a}}\,\mathtt{b} \quad \mathtt{eq} = \overline{\mathtt{a} \oplus \mathtt{b}} \quad \mathtt{gt} = \mathtt{a}\,\overline{\mathtt{b}}$

a bubble on an input inverts it before it enters the gate

We can always compute `lt` from `eq` and `gt` (as $\texttt{lt} = \overline{\texttt{gt}} \cdot \overline{\texttt{eq}}$), so we can simplify the circuit a bit:



To compare two (unsigned) n-bit numbers, the **first** place (from the left) where they differ decides.



comparing 11 and 9

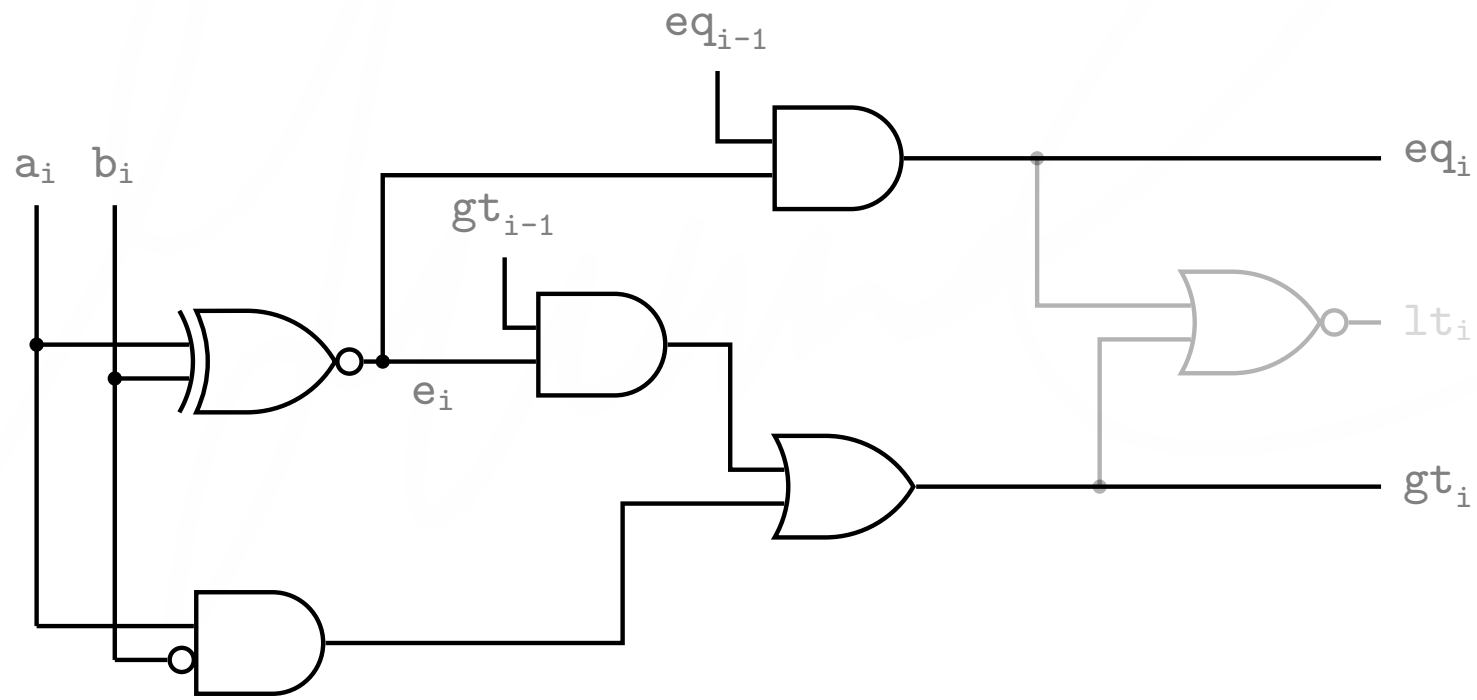
For signed numbers, only the top bit changes: a 1 there means *smaller*, so it swaps the roles of a and b.

We define $\texttt{gt}_i$, $\texttt{eq}_i$, and $\texttt{lt}_i$ to consider the *least-significant* i bits.

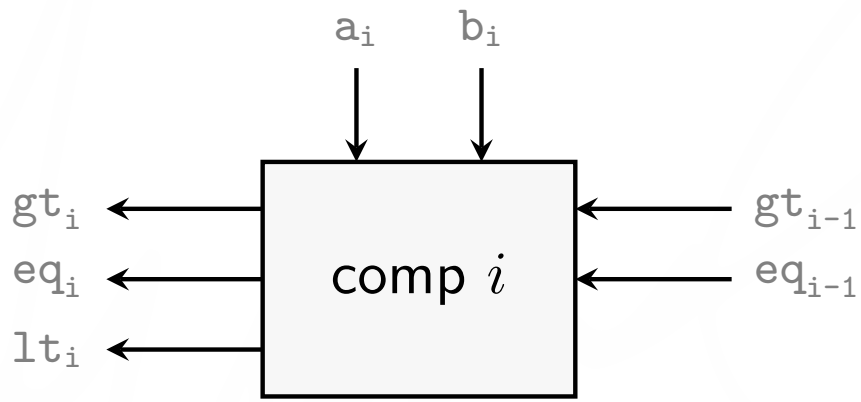
We conclude that gt_i should be 1 iff either:

- $a_i > b_i$, or
- $a_i = b_i$ and $gt_i = 1$

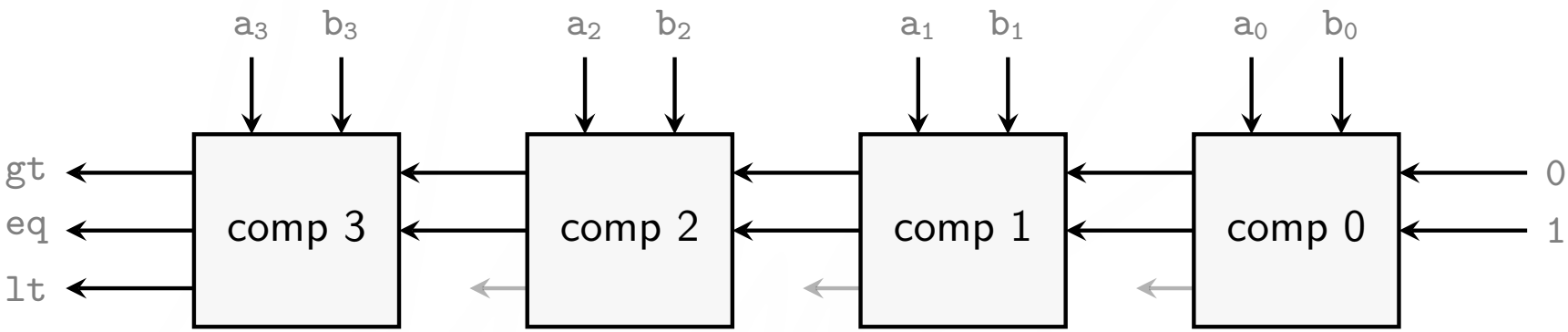
and eq_i should be 1 iff $a_i = b_i$, and $a_{i-1} = b_{i-1}$



The resulting circuit looks like this:

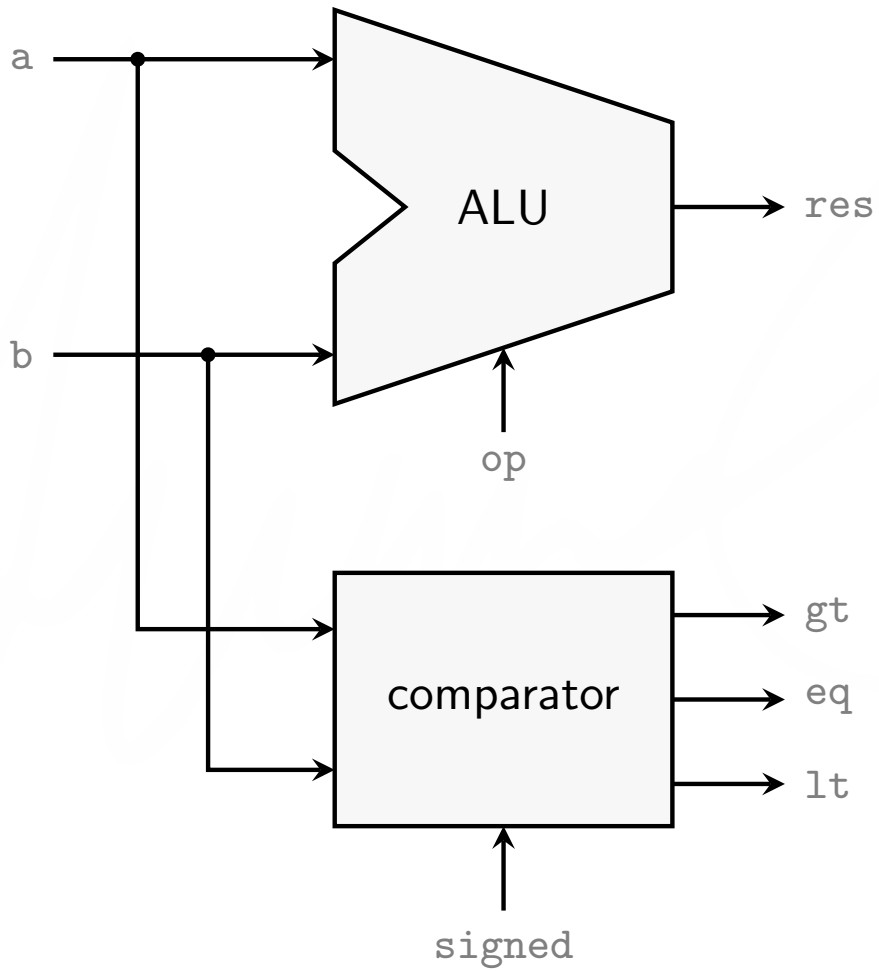


Each comp only needs its own bits and the result from the comp to its right, so we can chain n comps together to form an n-bit comparator:



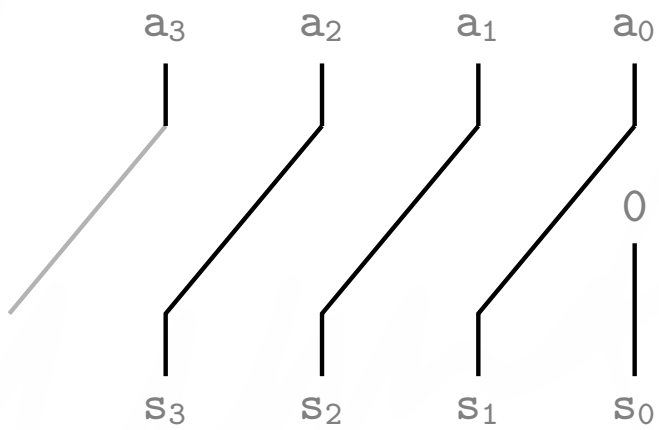
the chain starts at "equal" (eq $= 1$, gt $= 0$)

The comparator is its own block, **outside** the ALU. Both can be given the same inputs.



shifters

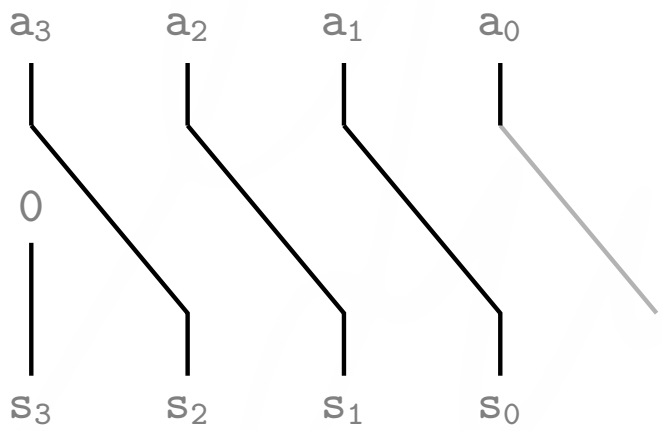
A **shifter** moves every bit one place to the left or right. Shifting **left** fills in a 0 on the right, and the top bit falls off.



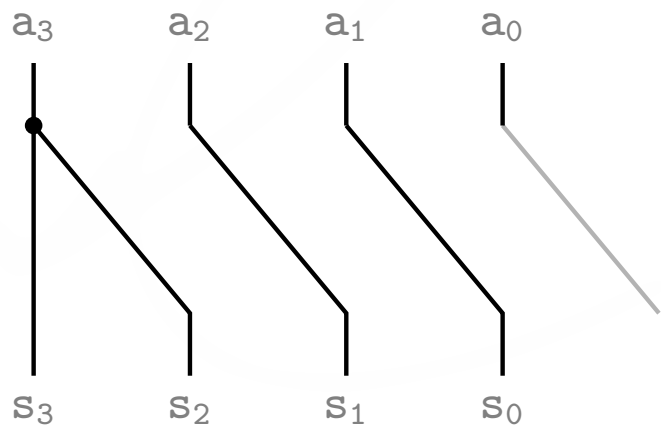
shifting left 1 bit

Shifting **right** drops the bottom bit, and there are two ways to fill in the top: a 0 (*logical*), or a copy of the sign bit (*arithmetic*), so negative numbers stay negative.

logical



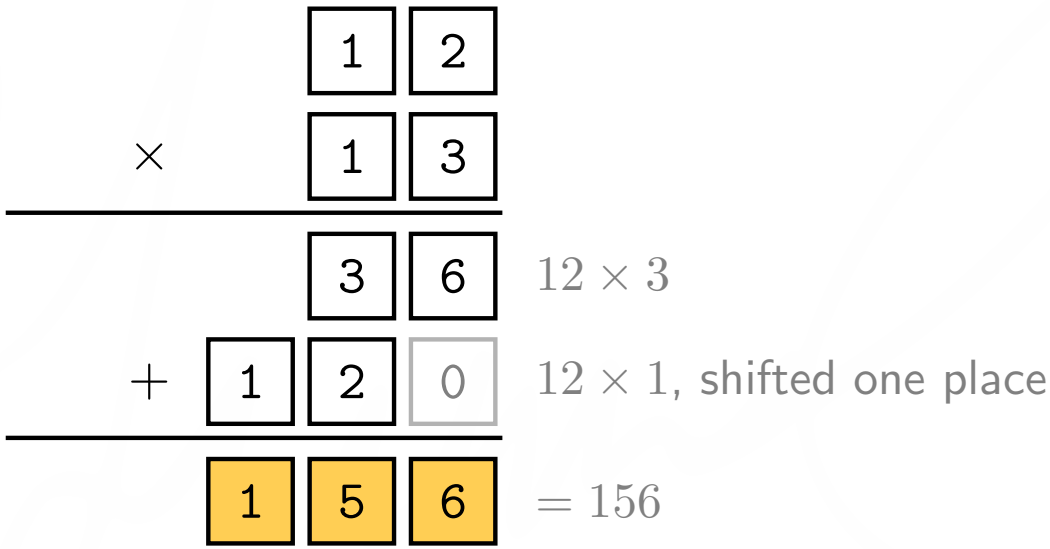
arithmetic



shifting right divides by 2 (rounding down)

multiplication

In decimal long multiplication, we multiply by one digit at a time, shift each row one place further left, and add the rows.

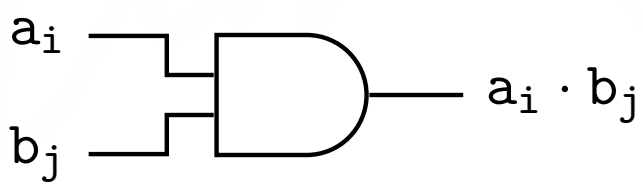


binary works the same way, with much simpler rows

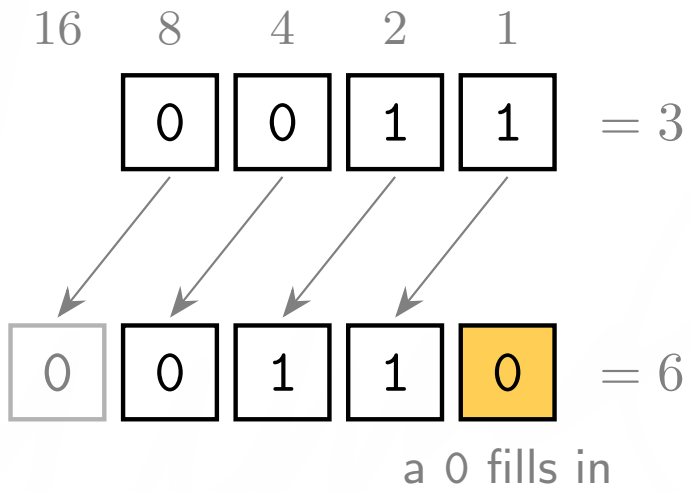
Every binary digit is 0 or 1, so each row is either a copy of the first number or all zeros.

1	0	1	1	× 1	=	1	0	1	1
1	0	1	1	× 0	=	0	0	0	0

Each bit of a row is 1 only when both bits are 1, which is just an AND gate.

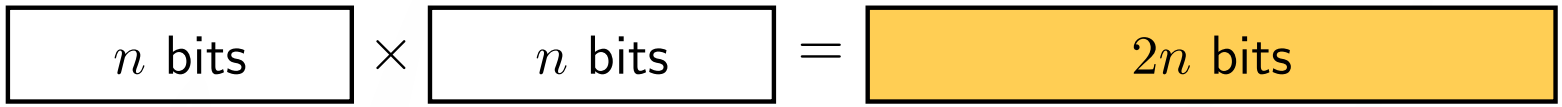


Shifting every bit one place left moves it to a weight twice as large, so the value doubles.

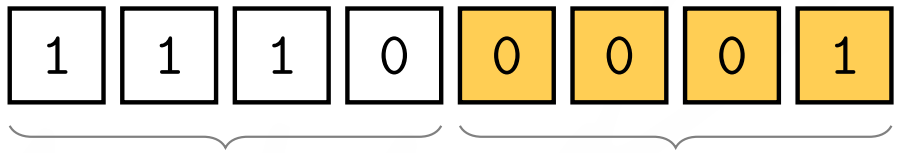


shifting left by k places multiplies by 2^k

The product of two n-bit numbers can need $2n$ bits.



$15 \times 15 = 225 =$

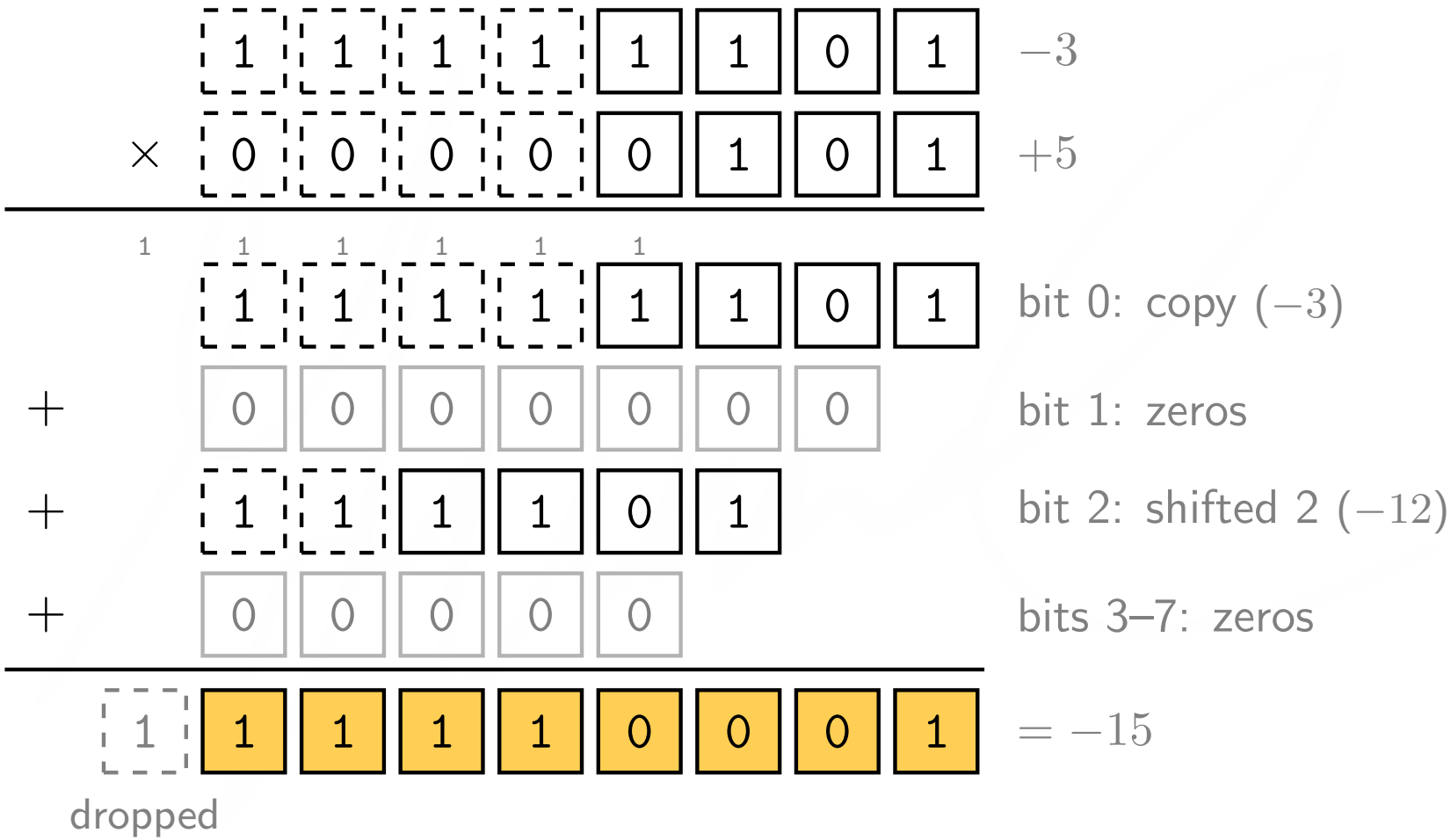


lost if we keep only 4 bits

$0001 = 1$, not 225

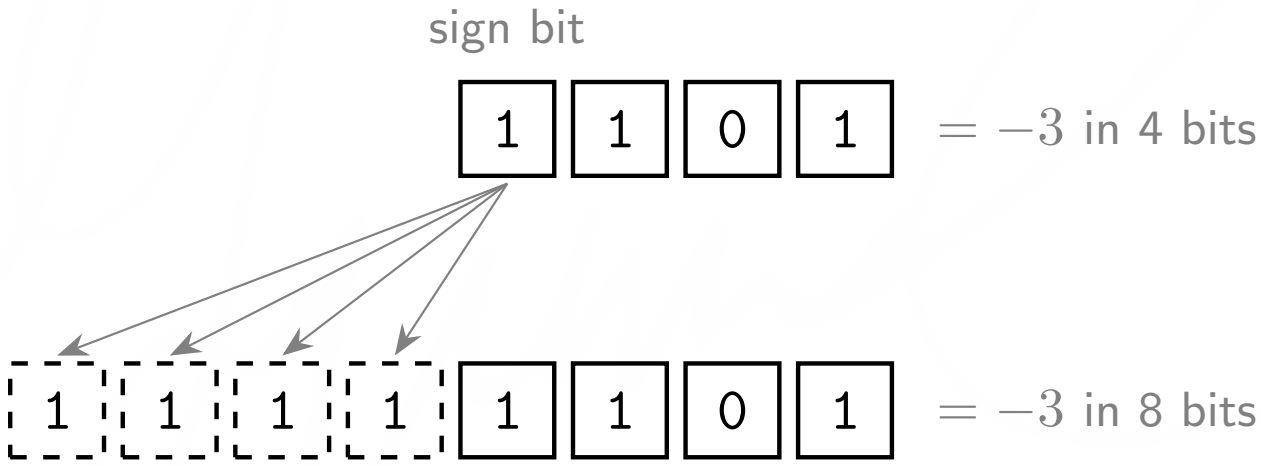
$(2^n - 1)^2 < 2^{2n}$, so $2n$ bits always suffice

To multiply (-3) by 5: first extend both inputs to 8 bits, then each row is a shifted copy of the top number.



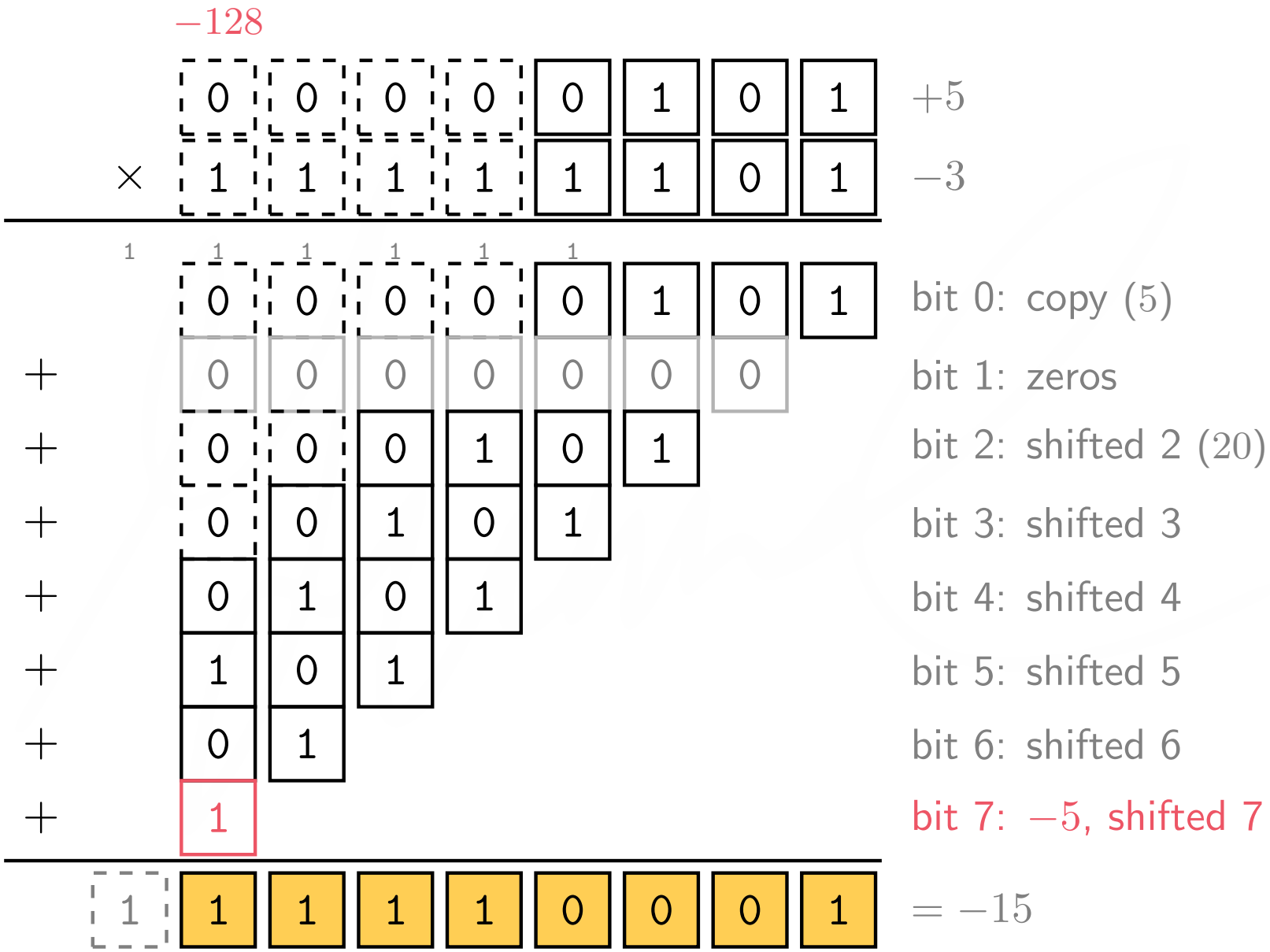
Unsigned numbers can be widened by adding 0s.

To widen a signed number without changing its value, we copy its sign bit into the new places.



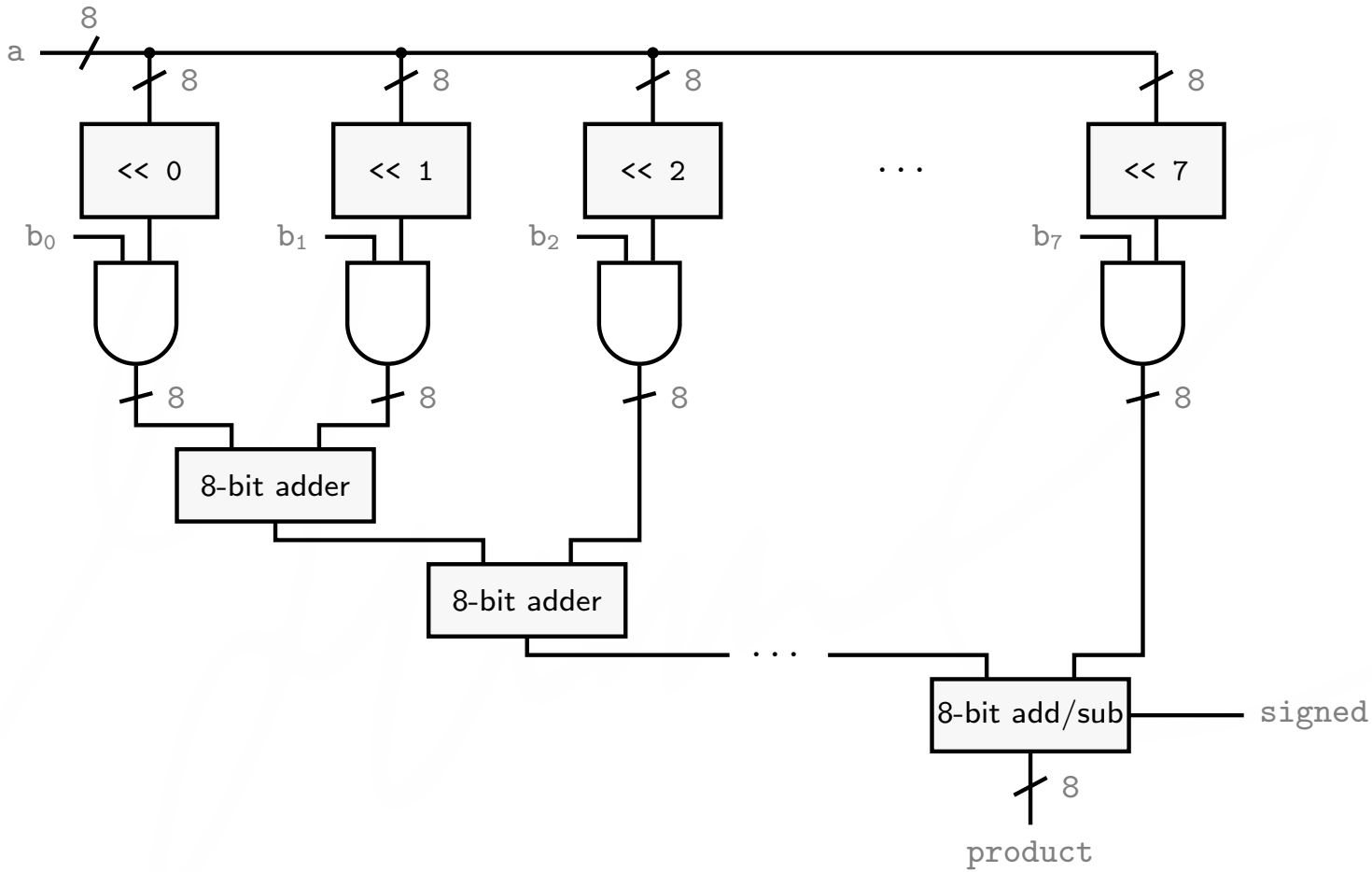
filling with zeros instead would give 00001101, which is 13

To multiply 5 by (-3), we use the same process, but subtract the last row.



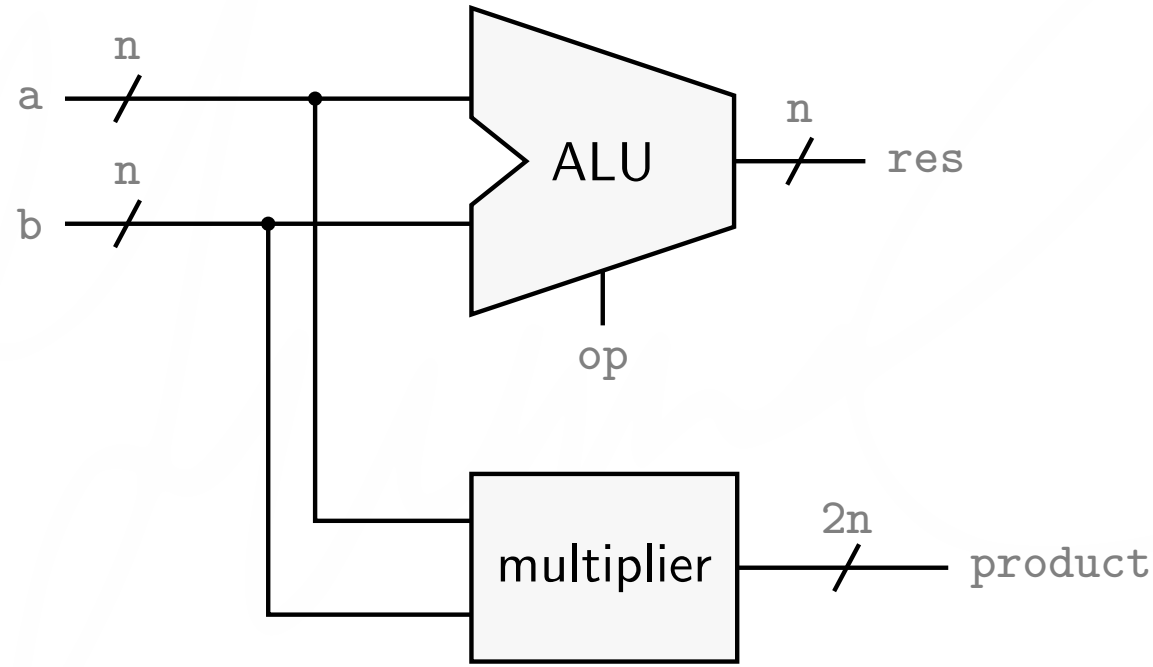
multipliers

In hardware, each row is a, shifted and ANDed with one bit of b.



A chain of 8-bit adders sums the rows; for signed numbers, the last one subtracts instead.

The multiplier usually sits **beside** the ALU rather than inside it: it is much larger, much slower, and its result is $2n$ bits wide.



We now have a way to represent numbers and compute with them. Next, we need a way to store them (i.e., memory).